

Biblioteca SPI para Microcontroladores AVR

1. Introdução

Esta biblioteca provê uma camada de abstração de hardware (HAL) para o periférico SPI, focando em comunicação assíncrona, segurança de memória (anti-overflow) e modularidade por injeção de pinos.

2. Estrutura de Arquivos

A biblioteca é composta por quatro arquivos fundamentais:

- **spi_types.h**: Definições de tipos, enums de status, structs e de hardware.
- **spi_internal.h**: Protótipos de funções de baixo nível.
- **spi.h**: API Pública.
- **spi.c**: Implementação da lógica.

3. Arquitetura do Sistema

A biblioteca utiliza um modelo de **Buffer Circular Duplo** (TX e RX) operando em regime de **Interrupção**.

Fluxo de Dados: No SPI, cada bit que sai pelo MOSI força a entrada de um bit pelo MISO simultaneamente. A biblioteca gerencia isso automaticamente: ao enviar um byte da fila TX, ela reserva o byte recebido no buffer RX.

3.1. Configuração e Inicialização

A configuração é feita via `spi_config_t`. A grande vantagem é a Injeção de Pinos, que permite definir o hardware sem alterar o código da biblioteca.

Exemplo de Configuração Master (125 kHz)

```
spi_config_t cfg = {
    .mode      = SPI_MODE_MASTER,
    .clock_div  = SPI_CLOCK_DIV_128, // Fosc/128
    .cpol      = 0,                  // Clock Idle em LOW
    .cpha      = 0,                  // Amostra na borda de subida
    .bit_order  = SPI_ORDER_MSB_FIRST,
    .use_interrupt = true,           // Habilita modo assíncrono
    .rx_callback = meu_callback,    // Função opcional (ou NULL)
    .pins = {
        .port = &PORTB,             // Porta base
        .mosi_pin = PB3, .miso_pin = PB4,
        .sck_pin  = PB5, .ss_pin  = PB2
    }
};
spi_init(&cfg);
```

3.2. Principais definições de tipos e Parâmetros

Para garantir a legibilidade, a biblioteca utiliza tipos enumerados. Abaixo, os valores que podem ser passados nas funções de configuração.

spi_mode_t (Modo de Operação)

Define o papel do microcontrolador no barramento.

- SPI_MODE_MASTER: O AVR gera o sinal de clock (SCK) e controla o Chip Select.
- SPI_MODE_SLAVE: O AVR aguarda o clock e o sinal de SS externo.

spi_clock_div_t (Divisores de Clock)

Define a velocidade do barramento (Baseado no F_CPU).

- SPI_CLOCK_DIV_2: Velocidade máxima ($F_{osc} / 2$). Ex: 8 MHz @ 16MHz.
- SPI_CLOCK_DIV_4: $F_{osc} / 4$. Ex: 4 MHz @ 16MHz.
- SPI_CLOCK_DIV_16: $F_{osc} / 16$. Ex: 1 MHz @ 16MHz.
- SPI_CLOCK_DIV_64: $F_{osc} / 64$. Ex: 250 kHz @ 16MHz.
- SPI_CLOCK_DIV_128: Velocidade mínima. Ex: 125 kHz @ 16MHz.

spi_bit_order_t (Ordem dos Bits)

- SPI_ORDER_MSB_FIRST: Envia o bit mais significativo (D7) primeiro. (Padrão industrial).
- SPI_ORDER_LSB_FIRST: Envia o bit menos significativo (D0) primeiro.

4. Guia de Funções (API Pública)

Gerenciamento de Dispositivos

- spi_register_slave(id, port, pin): Mapeia um Slave para um ID (0-3). Configura o pino como saída HIGH.

Parâmetros:

id: 0 a 3 (definido por SPI_MAX_SLAVES).

port: Endereço da porta (ex: &PORTB, &PORTC).

pin: Número do pino (0 a 7).

- `spi_select_device(id)`: Coloca o pino CS do Slave em LOW. Importante: Chame antes de enviar dados.

Uso: Obrigatório antes de chamar `spi_write_async` se o ID atual for diferente do anterior.

- `spi_deselect_device()`: Coloca o CS do Slave atual em HIGH.
- `void spi_set_clock(spi_clock_div_t new_div)`: Altera a velocidade do barramento em tempo de execução de forma atômica.

Parâmetro: Um dos valores de `spi_clock_div_t`.

Nota: Aguarda automaticamente o fim de qualquer transmissão antes de aplicar.

Transmissão de Dados

- `spi_write_async(byte)`: Coloca um byte na fila e retorna imediatamente. O hardware enviará o dado em background.

Retorno: **true** se houver espaço no buffer; **false** em caso de **Overflow**.

- `spi_write_multi(buffer, len, keep_cs)`: Envia um array. Se `keep_cs` for true, o CS não sobe entre os bytes (essencial para memórias e DACs).

`buffer`: Ponteiro para o array de dados.

`len`: Quantidade de bytes.

`keep_cs`: **true** - mantém o CS baixo (para memórias/DACs); **false** - libera o CS após o último byte.

- `spi_transfer_blocking(byte)`: Modo simplificado. Trava a CPU até o byte ser enviado. Útil para debug ou setups simples.

Recepção e Status

- `spi_rx_available()`: Retorna quantos bytes novos chegaram do Slave.
- `spi_rx_read(&byte)`: Retira o byte mais antigo do buffer RX. Retorna false se o buffer estiver vazio.

Retorno: **true** se havia dado para ler; **false** se o buffer RX estiver vazio.

- `spi_is_busy()`: Retorna true se o hardware ainda estiver transmitindo ou se houver algo na fila TX.

4.1. Callbacks e ISR

O **Callback** é uma função que você escreve no seu `main.c`, mas que a biblioteca executa "dentro" da interrupção sempre que um byte novo chega.

Exemplo de Callback

```
// Esta função roda IMEDIATAMENTE após o fim de cada byte recebido
void meu_callback(uint8_t dado_recebido, bool overflow) {
    if (overflow) {
        // Alerta: o buffer RX encheu e perdemos dados!
    }
    // Exemplo Slave: responder imediatamente com o inverso do que recebeu
    if (modo_slave) {
        spi_slave_preload(~dado_recebido);
    }
}
// Registro na inicialização:
cfg.rx_callback = minha_funcao_evento;
```

Aviso: Nunca use `_delay_ms()` ou funções de UART (`printf`) dentro de um callback. Isso trava o barramento SPI.

5. Conclusão

Essa biblioteca foi projetada para ser:

- **Fácil de usar** (API intuitiva)
- **Flexível** (Pode ser usada em diferentes famílias de microcontroladores AVR)

Há exemplos práticos completos de uso da biblioteca na pasta `/examples/`.

Boa programação e bons projetos!

Tiago Henrique dos Santos

Contatos:

Canal do YouTube: <https://www.youtube.com/channel/UCiEwgsH0HEj3P2aAl-Sozw>

Instagram: <https://www.instagram.com/cienciaeletrica/>

E-mail: contato.cienciaeletrica@gmail.com